

Figure 1: Hierarchy information network

once a day; so there is no point in requesting for a delivery status online from the headquarters database twice or thrice a day. This would help save the computational resources of the central database and perform an export function into an HTML-page at the least-loaded time—for example, at night. Besides, the security policy would prohibit one from having direct access into this unified database (Base3). There is the possibility of connecting to PostgreSQL databases at the levels of Base1 and Base2 only. So the question is—how to get all portions of equal data but from different sources? This can be easily done just by having one common bit of information replicated everywhere, such as a unique identifier for each product, which can be found in the SQL database as well as in the HTML page.

Okay, so let's do it. First, we need to make a general enquiry, which would take the values from the PostgreSQL database and would print two dates of a product's passage through all warehouses in the chain. To do this, you need to prepare a list of all departments first (this information is present in the database too). Then display the products that belong to particular departments. It is quite clear that information in all databases should be the same—the only difference will be in the arrival date/time at every warehouse and the status of the product (for example, DELIVERED/ NOT_DELIVERED/DELAYED/ BROKEN, etc).

The best option for writing the enquiry code is to choose Apache Tomcat as the application server; for the computation part, use JSP-code with JSTL-inlets—no more and no less.

I don't want to go into the installation and configuration of Tomcat and the architecturing of a database - let's assume you already know it. I propose to take a look at the code that follows and try to understand how it functions.

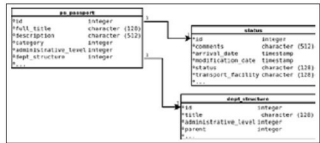


Figure 2: Tables' structures

Lines 1 - 3 declare that an HTML document with UTF8 encoding will be generated, which is quite a standard feature. Line 4 states that the variables in this page have a scope of one Web session. Lines 5 - 6 include support for JSTL-extensions; in our case this will be the functionality responsible for SQL-queries and core-functions, and the corresponding operators must go as clauses `<sql:action>` `</sql:action>`, `<c:action>` `</c:action>`. The following two lines, 7 - 8, are used to declare variables that will then be used during processing — `number` and `idDept`. The first is a products counter in every department. It is convenient to browse a list; besides, you see clearly how many goods per department are handled. The second variable is just an ID for the department.

Next, in Lines 9 - 11, we begin to prepare a casual HTML page with different attributes (like title, head, and body). The following few lines, namely 12 - 18, have an SQL-query that pools data directly from the database pool `MainPool_Level_1` (more on it later) into a variable `dept_names`. Lines 19 - 24 form a special HTML section, `SELECT`, that holds the names of departments and their IDs. You can choose a particular department by name. After that, this page will be automatically refreshed and supplied with a parameter (`id`), which is essentially a numerical value for the chosen department.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

3 <%@ page contentType="text/html; charset=UTF-8" %>
4 <%@ page session="true" %>
5 <%@ taglib uri="http://java.sun.com/jsp/jstl/sql"
  prefix="sql" %>
6 <%@ taglib uri="http://java.sun.com/jsp/jstl/core"
  prefix="c" %>

7 <%! Integer number = 1; %>
8 <%! String idDept = null; %>

9 <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en"
  lang="en">
10 <head><title>Products' within all warehouses (realtime)</
```